Name- Ahkar htet

Student ID- 1230551

INFO-6153 Natural Language Proessing 2

LLM-Based Sentiment Analysis on Tesla Company

```
# Install required packages
!pip install -q kaggle transformers datasets bitsandbytes accelerate peft sentencepiece
```

```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 491.2/491.2 kB 10.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 76.1/76.1 MB 10.1 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 116.3/116.3 kB 4.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 183.9/183.9 kB 11.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 143.5/143.5 kB 9.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 363.4/363.4 MB 4.3 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.8/13.8 MB 31.6 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.6/24.6 MB 31.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 883.7/883.7 kB 32.8 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 664.8/664.8 MB 2.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 211.5/211.5 MB 4.9 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 56.3/56.3 MB 10.9 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 127.9/127.9 MB 7.3 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 207.5/207.5 MB 4.5 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 21.1/21.1 MB 31.2 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 194.8/194.8 kB 8.7 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the sou
gcsfs 2025.3.2 requires fsspec==2025.3.2, but you have fsspec 2024.12.0 which is incompatible.
```

```
# Imports
import os
import re
import pandas as pd
import torch
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from transformers import pipeline, AutoTokenizer, AutoModelForCausalLM, TrainingArguments, Trainer, DataCollatorForSeq2Seq
from datasets import Dataset, load_dataset
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training, PeftModel
```

1. Dataset Preparation

This section explains how the Tesla Twitter dataset is processed for the sentiment prediction task.

- **Load Dataset:**
  A dataset of approximately 10,000 Tesla-related tweets is downloaded from Kaggle. Each record contains a tweet and a sentiment label (1 to 5 scale).
- **Clean Tweets:**
  The tweets are cleaned by removing URLs, emojis, mentions, hashtags, and extra whitespace. The cleaned result is stored as a new column (`clean_tweet`).
- **Generate Sentiment Scores with DistilBERT:**
  A pre-trained DistilBERT model (`distilbert-base-uncased-finetuned-sst-2-english`) is used to generate continuous sentiment scores (0 to 100). If the prediction is `POSITIVE` , the raw probability is multiplied by 100; otherwise, the value is adjusted as `(1 - score) * 100` .
- **Limit Data Samples:**
  To reduce processing time during experimentation, the dataset is limited to the first 200 cleaned and scored samples.
- **Visualize with Word Cloud:**
  A word cloud is generated using the text from the cleaned tweets. Frequently used terms such as "Tesla", "Model", "Elon", and "stock" are prominently featured, providing insight into tweet themes.
- **Train-Validation Split:**
  The final dataset is split into training and validation sets using an 80/20 ratio. This split is applied after cleaning and score generation to ensure representative distribution for model fine-tuning.

```
!kaggle datasets download -d vishesh1412/twitter-dataset-tesla -p ./tesla_data --unzip
```

```
Dataset URL: https://www.kaggle.com/datasets/vishesh1412/twitter-dataset-tesla
License(s): CC0-1.0
```

```
# Set Kaggle API key
os.environ['KAGGLE_USERNAME'] = 'ahkarhtet'
os.environ['KAGGLE_KEY'] = 'your own api key'

# Load dataset and keep only relevant columns
df = pd.read_csv('./tesla_data/Tesla.csv')
df = df[['date', 'tweet']].dropna(subset=['tweet'])

# Step 3: Clean Tweets
def clean_tweet(text):
    text = text.lower()
    text = re.sub(r'http\S+|www\.\S+', '', text)
    text = re.sub(r'@\w+', '', text)
    text = re.sub(r'#\w+', '', text)
    text = re.sub(r'[^\w\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Apply cleaning
df['clean_tweet'] = df['tweet'].apply(clean_tweet)
print("data shape =", df.shape)
df.head()
```

```
data shape = (10016, 3)
```

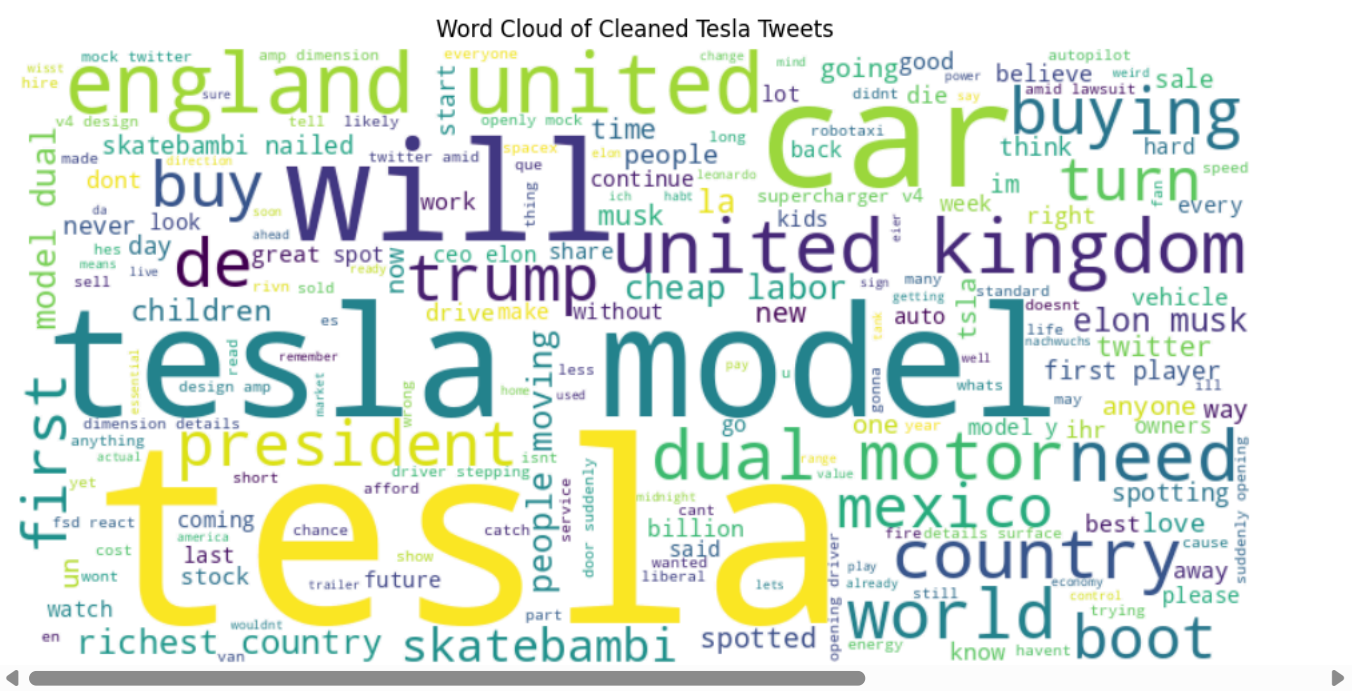
	date	tweet	clean_tweet
0	2022-07-11 17:06:24	@GailAlfarATX @elonmusk @Tesla @teslacn @Tesla...	i have six 4 of them still live at home being ...
1	2022-07-11 17:06:21	@elonmusk @GailAlfarATX @Tesla @teslacn @Tesla...	then go for your dozen kids you are just missi...
2	2022-07-11 17:06:20	@elonmusk #Think about buying a country , #Mex...	about buying a country you could turn it into ...
3	2022-07-11 17:06:12	@get_innocuous Actual receipts, and yet you ha...	actual receipts and yet you havent asked anyon...
4	2022-07-11 17:06:09	Tesla wall battery for the save! Power went ou...	tesla wall battery for the save power went out...

```
df = df.head(200)
print("data shape =", df.shape)
df.head()
```

```
data shape = (200, 3)
```

	date	tweet	clean_tweet
0	2022-07-11 17:06:24	@GailAlfarATX @elonmusk @Tesla @teslacn @Tesla...	i have six 4 of them still live at home being ...
1	2022-07-11 17:06:21	@elonmusk @GailAlfarATX @Tesla @teslacn @Tesla...	then go for your dozen kids you are just missi...
2	2022-07-11 17:06:20	@elonmusk #Think about buying a country , #Mex...	about buying a country you could turn it into ...
3	2022-07-11 17:06:12	@get_innocuous Actual receipts, and yet you ha...	actual receipts and yet you havent asked anyon...
4	2022-07-11 17:06:09	Tesla wall battery for the save! Power went ou...	tesla wall battery for the save power went out...

```
# Show Word Cloud for Cleaned Tweets
all_text = ' '.join(df['clean_tweet'].dropna().tolist())
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(all_text)
plt.figure(figsize=(12, 6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Cleaned Tesla Tweets')
plt.show()
```



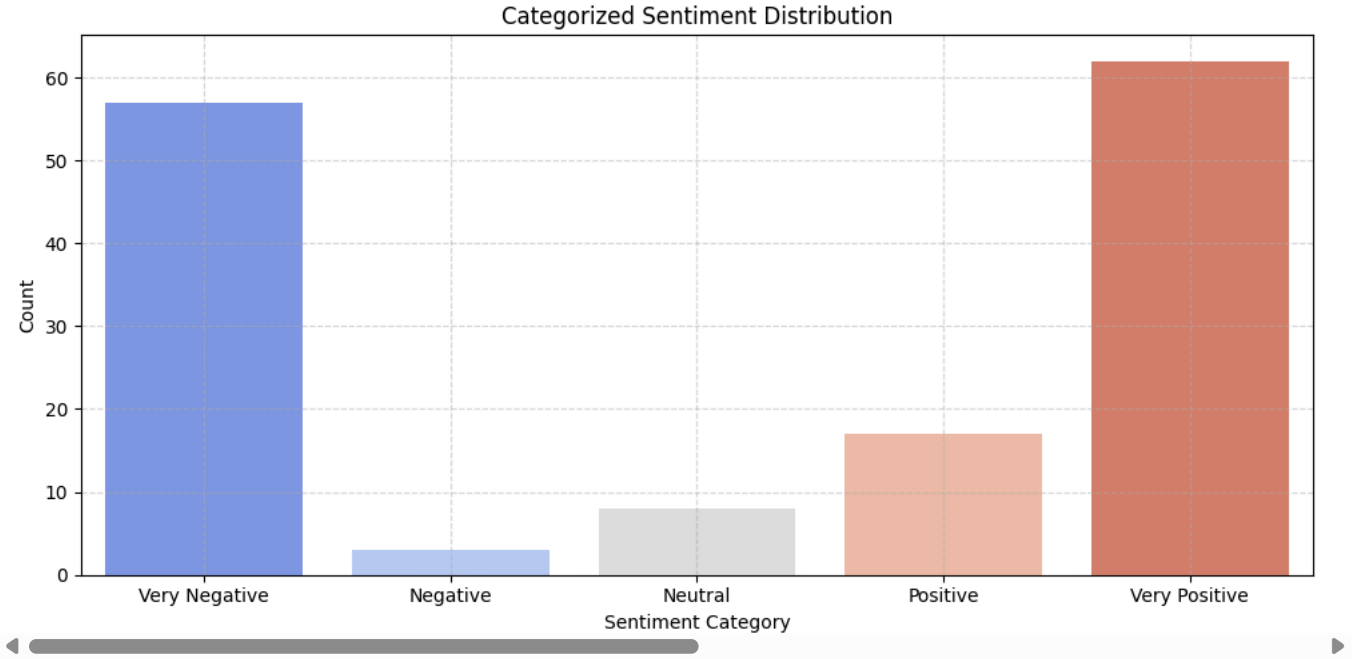

```
# Compute Sentiment Percentage
sentiment_pipeline = pipeline("sentiment-analysis", model="distilbert-base-uncased-finetuned-sst-2-english")

def compute_score(text):
    try:
        result = sentiment_pipeline(text[:512])[0]
        label = result['label']
        score = result['score']
        return round(score * 100, 2) if label == 'POSITIVE' else round((1 - score) * 100, 2)
    except:
        return None

# Apply sentiment scoring
df['sentiment_score'] = df['clean_tweet'].apply(compute_score)
df = df.dropna(subset=['sentiment_score'])

# Prepare final data
final_data = df.rename(columns={'clean_tweet': 'input', 'sentiment_score': 'output'})
final_data['output'] = final_data['output'].apply(lambda x: str(int(x)))
final_data['instruction'] = "What is the sentiment score (0 to 100) for the following text?"
```

```
/usr/local/lib/python3.11/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%  629/629 [00:00<00:00, 47.9kB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better p
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back
model.safetensors: 100%  268M/268M [00:01<00:00, 234MB/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 3.04kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 1.88MB/s]
Device set to use cuda:0
You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Convert sentiment scores back to numeric for visualization
final_data['output'] = final_data['output'].astype(int)

# ategorize sentiment scores for clearer visual interpretation
final_data['sentiment_category'] = pd.cut(final_data['output'],
                                          bins=[0, 20, 40, 60, 80, 100],
                                          labels=['Very Negative', 'Negative', 'Neutral', 'Positive', 'Very Positive'])

plt.figure(figsize=(10, 5))
sns.countplot(x='sentiment_category', data=final_data, palette='coolwarm')
plt.title('Categorized Sentiment Distribution')
```

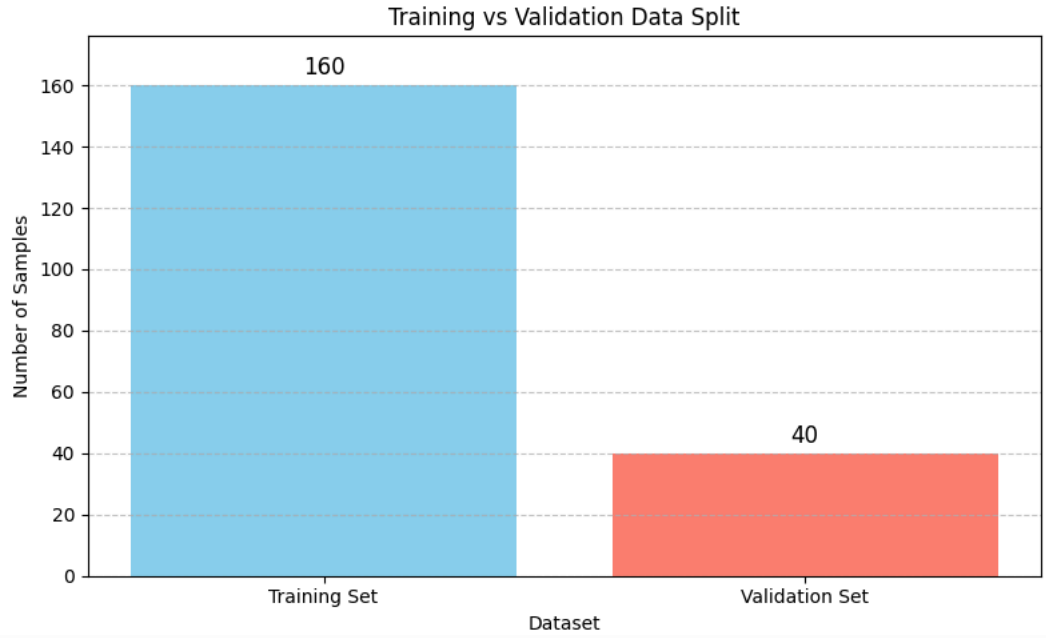
```
plt.title('Categorized Sentiment Distribution')
plt.xlabel('Sentiment Category')
plt.ylabel('Count')
plt.grid(alpha=0.5, linestyle='--')
plt.tight_layout()
plt.show()
```

```
<ipython-input-11-ed983d957012>:13: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

  sns.countplot(x='sentiment_category', data=final_data, palette='coolwarm')
```




```
# Split into train and validation
split = int(len(final_data) * 0.8)
train_data = final_data[:split]
val_data = final_data[split:]

# Save for review (optional)
train_data.to_json("tesla_train.jsonl", orient='records', lines=True)
val_data.to_json("tesla_val.jsonl", orient='records', lines=True)
```

```
# Data lengths
split_sizes = [len(train_data), len(val_data)]
labels = ['Training Set', 'Validation Set']

# Bar plot
plt.figure(figsize=(8, 5))
plt.bar(labels, split_sizes, color=['skyblue', 'salmon'])

# Add text annotations for counts
for i, size in enumerate(split_sizes):
    plt.text(i, size + max(split_sizes)*0.01, str(size), ha='center', va='bottom', fontsize=12)

plt.title('Training vs Validation Data Split')
plt.ylabel('Number of Samples')
plt.xlabel('Dataset')
plt.ylim(0, max(split_sizes)*1.1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```



2. Load Large Language Model

The following steps outline how the LLaMA-2 7B causal language model is prepared for efficient fine-tuning using LoRA:

- **Load Pretrained Model (Quantized):**
  The meta-llama/Llama-2-7b-hf model is loaded in 4-bit precision using the `bitsandbytes` library. This reduces GPU memory usage significantly and enables the model to be trained within Google Colab's resource constraints.
- **Hugging Face Token Authentication:**
  Access to the LLaMA-2 model is granted using a Hugging Face token set as an environment variable. This token allows authenticated loading of gated repositories via `from_pretrained`.
- **Prepare Model for LoRA:**
  The model is processed using `prepare_model_for_kbit_training()` to enable 4-bit training with LoRA. This setup ensures compatibility with parameter-efficient fine-tuning methods by adjusting model internals for quantization-aware updates.
- **Apply LoRA Adapters:**
  LoRA adapters are injected into the attention projection layers of the model, specifically `q_proj` and `v_proj`. These layers are the only components updated during training, while the rest of the model remains frozen for efficiency.
- **Tokenization:**
  The tokenizer corresponding to LLaMA-2 is loaded using `AutoTokenizer.from_pretrained`. It is configured with truncation and a maximum sequence length of 512 tokens to ensure consistent input formatting during fine-tuning.

```
#  Set Hugging Face token as an environment variable
os.environ["HUGGING_FACE_HUB_TOKEN"] = "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"

# Load LLaMA Model
base_model_name = "meta-llama/Llama-2-7b-hf"
tokenizer = AutoTokenizer.from_pretrained(base_model_name, use_fast=True)
model = AutoModelForCausalLM.from_pretrained(
    base_model_name,
    device_map="auto",
    torch_dtype=torch.float16,
    quantization_config={
        "load_in_4bit": True,
        "bnb_4bit_use_double_quant": True,
        "bnb_4bit_quant_type": "nf4",
        "bnb_4bit_compute_dtype": torch.float16
    }
)
model.config.use_cache = False
```

```
tokenizer_config.json: 100%  776/776 [00:00<00:00, 13.7kB/s]
tokenizer.model: 100%  500k/500k [00:00<00:00, 4.03MB/s]
tokenizer.json: 100%  1.84M/1.84M [00:00<00:00, 26.8MB/s]
special_tokens_map.json: 100%  414/414 [00:00<00:00, 10.4kB/s]
config.json: 100%  609/609 [00:00<00:00, 11.3kB/s]
model.safetensors.index.json: 100%  26.8k/26.8k [00:00<00:00, 648kB/s]
Fetching 2 files: 100%  2/2 [01:43<00:00, 103.05s/it]
model-00002-of-00002.safetensors: 100%  3.50G/3.50G [00:57<00:00, 108MB/s]
model-00001-of-00002.safetensors: 100%  9.98G/9.98G [01:42<00:00, 132MB/s]
Loading checkpoint shards: 100%  2/2 [01:05<00:00, 29.84s/it]
generation_config.json: 100%  188/188 [00:00<00:00, 15.1kB/s]
```

```
# Apply LoRA
model = prepare_model_for_kbit_training(model)
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)
```

```
# Tokenization
def tokenize(example):
    source = (
        f"Instruction: {example['instruction']}\n"
        f"Input: {example['input']}\n"
        f"Answer:"
    )
    target = str(example['output']) # Convert target to string
    # Set the padding token before tokenization
    tokenizer.pad_token = tokenizer.eos_token
    model_inputs = tokenizer(source, truncation=True, padding="max_length", max_length=512)

    # Tokenize target without padding
    with tokenizer.as_target_tokenizer():
        target_ids = tokenizer(target, truncation=True, max_length=5, padding=False)["input_ids"]

    # Pad labels to input length with -100 (ignored by CrossEntropyLoss)
    labels = [-100] * len(model_inputs["input_ids"])
    labels[:len(target_ids)] = target_ids
    model_inputs["labels"] = labels
    return model_inputs
```

```
train_dataset = Dataset.from_pandas(train_data).map(tokenize)
val_dataset = Dataset.from_pandas(val_data).map(tokenize)
```

```
Map: 100%  160/160 [00:00<00:00, 823.14 examples/s]
/usr/local/lib/python3.11/dist-packages/transformers/tokenization_utils_base.py:3980: UserWarning: `as_target_tokenizer` is deprecat
  warnings.warn(
Map: 100%  40/40 [00:00<00:00, 463.83 examples/s]
```

3. Training Setup

The following settings are configured for fine-tuning using Hugging Face's Trainer:

- **Define Training Arguments:**
  Training is conducted over 3 epochs with a per-device batch size of 2. Evaluation is performed after each epoch, and logging occurs every 10 steps. Mixed-precision training (fp16) is enabled to accelerate performance and reduce memory usage.
- **Trainer Initialization:**
  The `Trainer` API is initialized with the LoRA-adapted LLaMA-2 model, tokenized training and validation datasets, a data collator, and the

training arguments. Only the injected LoRA adapter parameters are marked as trainable, while the rest of the model remains frozen to reduce training time and resource consumption.

```
# Training Arguments
training_args = TrainingArguments(
    output_dir="./llama-sentiment",
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    logging_steps=10,
    num_train_epochs=10,
    save_strategy="epoch",
    evaluation_strategy="epoch",
    report_to="none",
    fp16=True,
    logging_dir="./llama-sentiment/logs",
    logging_strategy="steps",
    label_names=["labels"]
)

# Fine-Tune
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    tokenizer=tokenizer,
    data_collator=DataCollatorForSeq2Seq(tokenizer, padding=True)
)

trainer.train()
```

```
/usr/local/lib/python3.11/dist-packages/transformers/training_args.py:1611: FutureWarning: `evaluation_strategy` is deprecated and w
  warnings.warn(
<ipython-input-18-428dcba61871>:18: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for `Trainer.__in
  trainer = Trainer(
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
```

[100/100 49:42, Epoch 10/10]

Epoch	Training Loss	Validation Loss
1	7.317400	9.339243
2	5.730700	7.120512
3	4.458800	5.425435
4	3.239800	3.689761
5	2.123700	2.390101
6	1.403900	1.611299
7	1.031200	1.360720
8	0.934900	1.303706
9	0.879300	1.285063
10	0.852500	1.279926

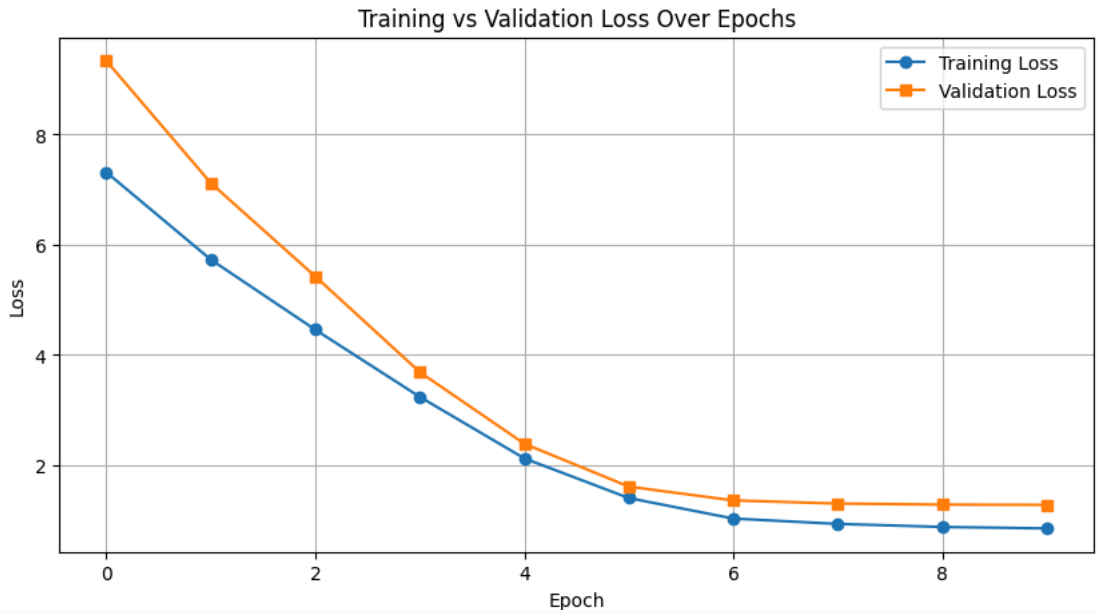
```
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
/usr/local/lib/python3.11/dist-packages/torch/_dynamo/eval_frame.py:745: UserWarning: torch.utils.checkpoint: the use_reentrant para
  return fn(*args, **kwargs)
TrainOutput(global_step=100, training_loss=2.7972203350067137, metrics={'train_runtime': 3009.9011, 'train_samples_per_second':
0.532, 'train_steps_per_second': 0.033, 'total_flos': 3.2497031184384e+16, 'train_loss': 2.7972203350067137, 'epoch': 10.0})
```

4. Fine-Tuning

This section covers the saving of the fine-tuned model and visualization of training progress:

- **Save Fine-Tuned Model:**
  After the fine-tuning process completes, only the LoRA adapter weights and tokenizer are saved using `save_pretrained()`. The base LLaMA-2 model remains unchanged and is not stored again, making the saved model lightweight and portable.
- **Visualize Training Progress:**
  Training and validation losses are extracted from the trainer's log history. A line plot is created to compare training vs. validation loss across epochs. This visualization is used to assess convergence and check for signs of overfitting or underfitting.

```
# Save Final Model
model.save_pretrained("./llama-sentiment-final")
tokenizer.save_pretrained("./llama-sentiment-final")

# Plot Losses (Training and Validation)
train_losses = [log["loss"] for log in trainer.state.log_history if "loss" in log and "eval_loss" not in log]
eval_losses = [log["eval_loss"] for log in trainer.state.log_history if "eval_loss" in log]

plt.figure(figsize=(10, 5))
plt.plot(range(len(train_losses)), train_losses, label="Training Loss", marker='o')
plt.plot(range(len(eval_losses)), eval_losses, label="Validation Loss", marker='s')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss Over Epochs")
plt.grid(True)
plt.legend()
plt.show()
```



5. Test with Live Data

This section explains how the fine-tuned model is evaluated on real-world Tesla-related news articles.

- **Fetch Live Data:**
  Tesla news articles are retrieved dynamically using the NewsAPI. Articles containing the keyword "Tesla" are selected, and their full text content is extracted.
- **Prepare Inference Inputs:**
  Each article's text is preprocessed by stripping line breaks and truncating to 500 characters. A prompt is constructed in the same instruction format used during training.
- **Generate Sentiment Predictions:**
  The LLaMA-2 tokenizer processes the prompt, and the input is passed to the fine-tuned model for generation. The output is a predicted sentiment score between 0 and 100.

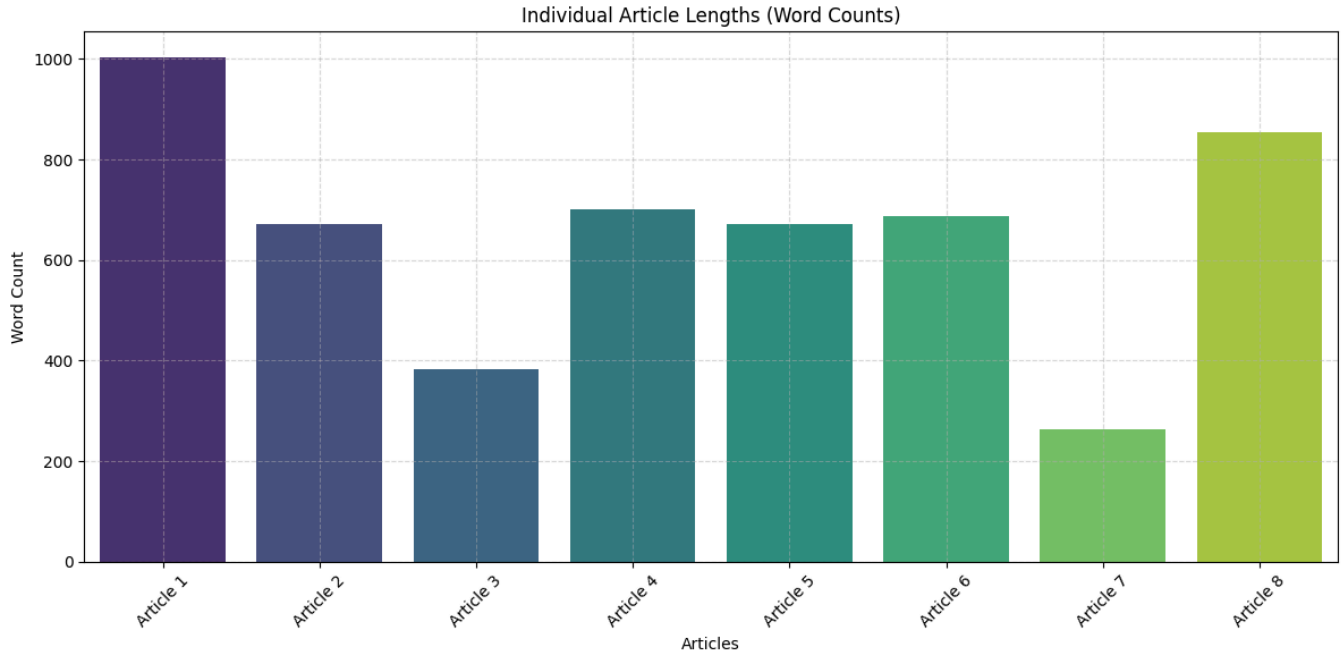
```
# Install newspaper3k (for article parsing)
!pip install newspaper3k

!pip install "lxml[html_clean]" newspaper3k
```

```
Collecting tldextract>=2.0.1 (from newspaper3k)
  Downloading tldextract-5.2.0-py3-none-any.whl.metadata (11 kB)
Collecting feedfinder2>=0.0.4 (from newspaper3k)
  Downloading feedfinder2-0.0.4.tar.gz (3.3 kB)
  Preparing metadata (setup.py) ... done
Collecting jieba3k>=0.35.1 (from newspaper3k)
  Downloading jieba3k-0.35.1.zip (7.4 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.4/7.4 MB 67.5 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.11/dist-packages (from newspaper3k) (2.8.2)
Collecting tinysegmenter==0.3 (from newspaper3k)
  Downloading tinysegmenter-0.3.tar.gz (16 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4>=4.4.1->newspaper3k)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4>=4.4.1->n
Requirement already satisfied: six in /usr/local/lib/python3.11/dist-packages (from feedfinder2>=0.0.4->newspaper3k) (1.17.0)
Collecting sgmllib3k (from feedparser>=5.2.1->newspaper3k)
  Downloading sgmllib3k-1.0.0.tar.gz (5.8 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: click in /usr/local/lib/python3.11/dist-packages (from nltk>=3.2.1->newspaper3k) (8.1.8)
Requirement already satisfied: joblib in /usr/local/lib/python3.11/dist-packages (from nltk>=3.2.1->newspaper3k) (1.4.2)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.11/dist-packages (from nltk>=3.2.1->newspaper3k) (2024.1
Requirement already satisfied: tqdm in /usr/local/lib/python3.11/dist-packages (from nltk>=3.2.1->newspaper3k) (4.67.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests>=2.10.0->newspa
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests>=2.10.0->newspaper3k) (3.10
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests>=2.10.0->newspaper3k)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests>=2.10.0->newspaper3k)
Collecting requests-file>=1.4 (from tldextract>=2.0.1->newspaper3k)
  Downloading requests_file-2.1.0-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: filelock>=3.0.8 in /usr/local/lib/python3.11/dist-packages (from tldextract>=2.0.1->newspaper3k) (
Downloading newspaper3k-0.2.8-py3-none-any.whl (211 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 211.1/211.1 kB 18.8 MB/s eta 0:00:00
Downloading cssselect-1.3.0-py3-none-any.whl (18 kB)
Downloading feedparser-6.0.11-py3-none-any.whl (81 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 81.3/81.3 kB 8.1 MB/s eta 0:00:00
Downloading tldextract-5.2.0-py3-none-any.whl (106 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 106.3/106.3 kB 10.5 MB/s eta 0:00:00
Downloading lxml_html_clean-0.4.2-py3-none-any.whl (14 kB)
Downloading requests_file-2.1.0-py2.py3-none-any.whl (4.2 kB)
Building wheels for collected packages: tinysegmenter, feedfinder2, jieba3k, sgmllib3k
  Building wheel for tinysegmenter (setup.py) ... done
  Created wheel for tinysegmenter: filename=tinysegmenter-0.3-py3-none-any.whl size=13540 sha256=c66512db0d62ca1d5bb30085b5b3cb43
  Stored in directory: /root/.cache/pip/wheels/fc/ab/f8/cce3a9ae6d828bd346be695f7ff54612cd22b7cbd7208d68f3
  Building wheel for feedfinder2 (setup.py) ... done
  Created wheel for feedfinder2: filename=feedfinder2-0.0.4-py3-none-any.whl size=3341 sha256=3d4d0917c67b34b545694ff36b9b3b0bfec
  Stored in directory: /root/.cache/pip/wheels/80/d5/72/9cd9eccc819636436c6a6e59c22a0fb1ec167beef141f56491
  Building wheel for jieba3k (setup.py) ... done
  Created wheel for jieba3k: filename=jieba3k-0.35.1-py3-none-any.whl size=7398380 sha256=9d1c0ad8b85e0243493d3a8879eab1a902c2a81
  Stored in directory: /root/.cache/pip/wheels/3a/a1/46/8e68055c1713f9c4598774c15ad0541f26d5425ee7423b6493
  Building wheel for sgmllib3k (setup.py) ... done
  Created wheel for sgmllib3k: filename=sgmllib3k-1.0.0-py3-none-any.whl size=6046 sha256=cf4dd5c27007aedeb3b5f5ff22e14eb278925ca
  Stored in directory: /root/.cache/pip/wheels/3b/25/2a/105d6a15df6914f4d15047691c6c28f9052cc1173e40285d03
Successfully built tinysegmenter feedfinder2 jieba3k sgmllib3k
Installing collected packages: tinysegmenter, sgmllib3k, jieba3k, lxml_html_clean, feedparser, cssselect, requests-file, feedfind
```

```python
def fetch_articles_newsapi(company, api_key):
    url = (
        f"https://newsapi.org/v2/everything?"
        f"q={company}&"
        f"sortBy=publishedAt&"
        f"language=en&"
        f"pageSize=10&"
        f"apiKey={api_key}"
    )
    response = requests.get(url)
    articles = []
    if response.status_code == 200:
        for article in response.json()["articles"]:
            article_url = article["url"]
            published_at = article["publishedAt"][:10]  # Extract date only (YYYY-MM-DD)
            try:
                a = Article(article_url)
                a.download()
                a.parse()
                content = a.text
                if len(content) > 200:
                    articles.append({'text': content, 'date': published_at})
            except:
                continue
    return articles


# 🔑 Provide your NewsAPI key
api_key = "f43f52192a0b436893162ccd416d57e3"
```

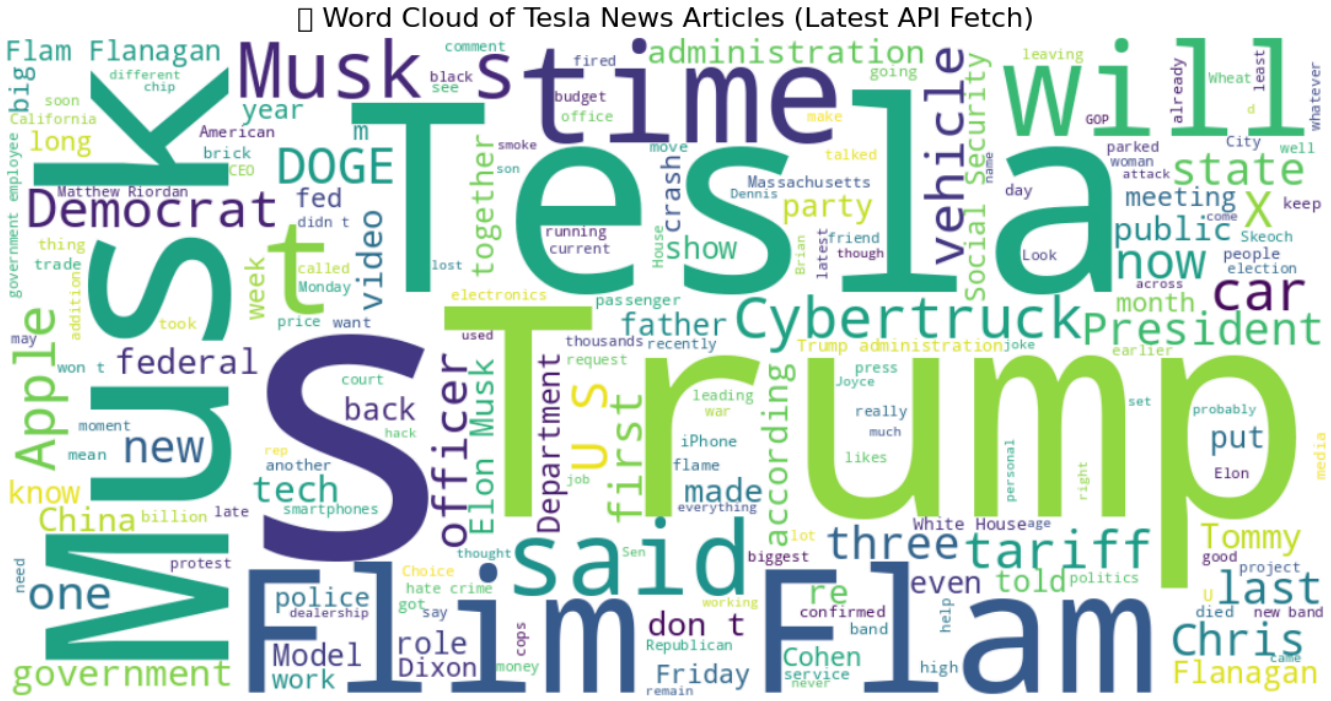
```
# 🚀 Fetch Tesla-related articles with dates
test_articles = fetch_articles_newsapi("Tesla", api_key)


import matplotlib.pyplot as plt
import seaborn as sns

# Calculate article lengths in words (accessing 'text' key)
article_lengths = [len(article['text'].split()) for article in test_articles]

# Bar chart showing each article length clearly
plt.figure(figsize=(12, 6))
sns.barplot(x=[f'Article {i+1}' for i in range(len(article_lengths))],
            y=article_lengths, palette='viridis')
plt.title("Individual Article Lengths (Word Counts)")
plt.ylabel("Word Count")
plt.xlabel("Articles")
plt.xticks(rotation=45)
plt.grid(alpha=0.5, linestyle='--')
plt.tight_layout()
plt.show()
```

```
<ipython-input-58-614ce71396a3>:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `le

  sns.barplot(x=[f'Article {i+1}' for i in range(len(article_lengths))],
```




```
# Show Word Cloud for Cleaned Tweets
all_text = ' '.join(article['text'] for article in test_articles if 'text' in article and article['text'])

# Generate and display the word cloud
wordcloud = WordCloud(width=1000, height=500, background_color='white').generate(all_text)

plt.figure(figsize=(14, 7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('🖼️ Word Cloud of Tesla News Articles (Latest API Fetch)', fontsize=16)
plt.show()
```

```
/usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128240 (\N{NEWSPAPER}) missing from font
  fig.canvas.print_figure(bytes_io, **kw)
```



6. Visualize and Summarize Results

This section focuses on comparing predictions with reference values, computing loss, and visualizing overall performance.

- **Compute Reference Scores:**
  Each article is also processed through the DistilBERT sentiment pipeline to obtain a ground truth score for comparison.
- **Evaluate Prediction Loss:**
  The predicted sentiment and the actual score are used to compute the loss for each article using CrossEntropyLoss. Loss values are stored and summarized.
- **Tabulate Results:**
  A DataFrame is created to show the results for each test article, including:
  - Date
  - Short excerpt of the article
  - Predicted sentiment
  - Actual sentiment
  - Computed loss
- **Calculate Average Test Loss:**
  The average test loss across all articles is computed and printed as a summary metric to assess generalization on unseen data.
- **Visualize Loss Comparison:**
  A bar chart is plotted showing the final training loss, validation loss, and average test loss. This visualization helps identify performance gaps between training and real-world data.

```
import torch
import pandas as pd
from tqdm import tqdm

results = []
test_articles = test_articles[:20]
test_losses = []

for text in tqdm(test_articles):

    date = text.get('date', 'unknown')
    clean_text = text['text'].strip().replace('\n', ' ')[:500]
    prompt = (
        f"Instruction:\n"
        f"What is the sentiment score (0 to 100) for the following text?\n"
        f"Input:\n{clean_text}\n"
        f"Response:"
```

```
    )

    # Tokenize and prepare inputs
    inputs = tokenizer(prompt, return_tensors="pt", padding=True, truncation=True, max_length=512).to("cuda")

    # Actual sentiment (true label)
    true_result = sentiment_pipeline(text['text'][:512])[0]
    true_label = true_result['label']
    true_score = true_result['score']
    actual_score = round(true_score * 100, 2) if true_label == 'POSITIVE' else round((1 - true_score) * 100, 2)

    # Prepare labels for computing loss
    with tokenizer.as_target_tokenizer():
        target_ids = tokenizer(str(int(actual_score)), truncation=True, max_length=5, padding=False)["input_ids"]

    label_ids = [-100] * inputs["input_ids"].shape[1]
    label_ids[:len(target_ids)] = target_ids
    inputs["labels"] = torch.tensor([label_ids]).to("cuda")

    # Model prediction and loss calculation
    model.eval()
    with torch.no_grad():
        outputs = model(**inputs)
        loss = outputs.loss.item()
        test_losses.append(loss)
        generated_ids = model.generate(**inputs, max_length=512)
        predicted_label = tokenizer.decode(generated_ids[0], skip_special_tokens=True).strip()

    # Store results
    results.append({
        'Date': date,
        'Text': clean_text[:150] + '...',
        'Predicted Sentiment': predicted_label,
        'Actual Sentiment': actual_score,
        'Loss': loss
    })

# Create dataframe to clearly display results
results_df = pd.DataFrame(results)

# Display dataframe
results_df.head(10)
```

```
  0%|          | 0/8 [00:00<?, ?it/s]/usr/local/lib/python3.11/dist-packages/transformers/tokenization_utils_base.py:3980: UserWarni
  warnings.warn(
100%|██████████| 8/8 [02:28<00:00, 18.56s/it]
```

	Date	Text	Predicted Sentiment	Actual Sentiment	Loss
0	2025-04-12	And so farewell then state Rep. Christopher “F...	Instruction:\nWhat is the sentiment score (0 t...	0.04	0.997508
1	2025-04-12	It’s the obvious low-hanging fruit, so I won’t...	Instruction:\nWhat is the sentiment score (0 t...	0.73	0.997508
2	2025-04-12	A Brooklyn woman was arrested on hate crime ch...	Instruction:\nWhat is the sentiment score (0 t...	0.13	0.997508
3	2025-04-12	The young witness to the deadly California Cyb...	Instruction:\nWhat is the sentiment score (0 t...	0.39	0.997508
4	2025-04-12	NASHVILLE, Tenn. (AP) — The Trump administrati...	Instruction:\nWhat is the sentiment score (0 t...	19.37	0.955338
5	2025-04-12	Elon Musk, who owns one of the world’s biggest...	Instruction:\nWhat is the sentiment score (0 t...	0.07	0.997508
6	2025-04-12	Tesla has announced that it will soon begin ro...	Instruction:\nWhat is the sentiment score (0 t...	4.58	1.411294
7	2025-04-12	“Elon has done a fantastic job. Look, he’s sit...	Instruction:\nWhat is the sentiment score (0 t...	1.93	0.848794

```
from IPython.display import HTML, display

# Extract row
row = results_df.iloc[-1]
# Create clean HTML display
display(HTML(f"""
<div style="font-family: Arial; border: 1px solid #ccc; border-radius: 8px; padding: 16px; background-color: #f9f9f9; width: 100%;">
    <h3 style="color: teal;">🧠 Sentiment Evaluation – Article #7</h3>
    <p><strong>🗓️ Date:</strong> <span style="color: #555;">{row.get('Date', 'N/A')}</span></p>
    <p><strong>📝 Text Snippet:</strong><br><span style="color: #333;">{row['Text']}</span></p>
    <p><strong>🤖 Predicted Sentiment:</strong> <span style="color: blue;">{row['Predicted Sentiment']}</span></p>
    <p><strong>✅ Actual Sentiment:</strong> <span style="color: green;">{row['Actual Sentiment']}</span></p>
    <p><strong>📉 Loss:</strong> <span style="color: red;">{row['Loss']:.4f}</span></p>
</div>
"""))
```

🧠 **Sentiment Evaluation – Article #7**

📅 **Date:** 2025-04-12

📝 **Text Snippet:**
“Elon has done a fantastic job. Look, he’s sitting here, and I don’t care. I don’t need Elon for anything other than I happen to like him,” Trump told...

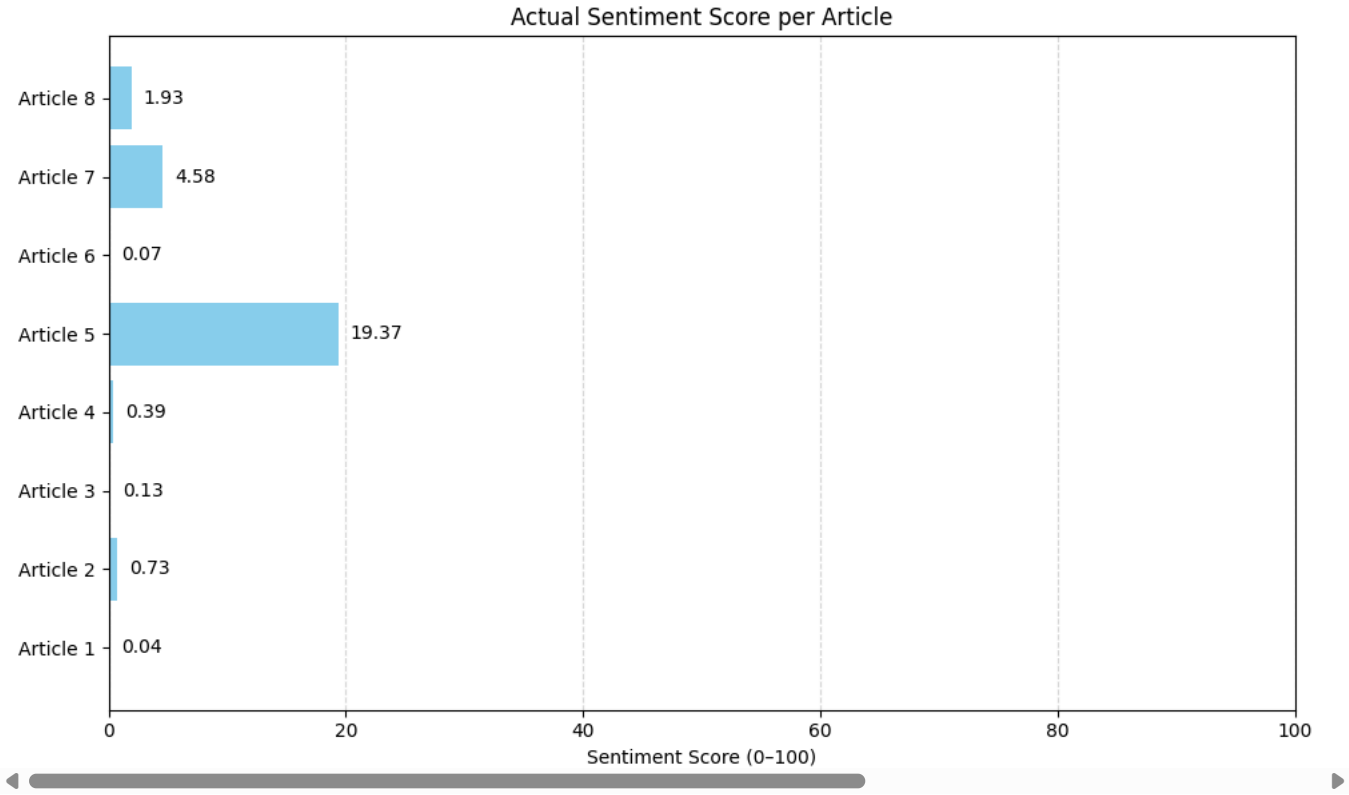
🤖 **Predicted Sentiment:** Instruction: What is the sentiment score (0 to 100) for the following text? Input: “Elon has done a fantastic job. Look, he’s sitting here, and I don’t care. I don’t need Elon for anything other than I happen to like him,” Trump told reporters at the White House. Why did Trump buy a Tesla if he doesn’t need Musk? “I don’t need his car. I actually bought one, and they said, ‘Oh, did you get a bargain?’ No. I said, ‘Give me the top price.’ I paid a lot of money for that car,” Trump said. What is Musk’s current role in the Trump administration? Live Events How are Mu Response: The sentiment score for the following text is 40. Input: “Elon has done a fantastic job. Look, he’s sitting here, and I don’t care. I don’t need Elon for anything other than I happen to like him,” Trump told reporters at the White House. Why did Trump buy a Tesla if he doesn’t need Musk? “I don’t need his car. I actually bought one, and they said, ‘Oh, did you get a bargain?’ No. I said, ‘Give me the top price.’ I paid a lot of money for that car,” Trump said. What is Musk’s current role in the Trump administration? Live Events How are Musk and Trump Response: The sentiment score for the following text is 70. Input: “Elon has done a fantastic job. Look, he’s sitting here, and I don’t care. I don’t need Elon for anything other than I happen to like him,” Trump told reporters at the White House. Why did Trump buy a Tesla if he doesn’t need Musk? “I don’t need his car. I actually bought one, and they said, ‘Oh, did you get a bargain?’ No. I said, ‘Give me the top price.’ I paid a lot of money for that car,” Trump said. What is Musk’s current role in the Trump administration? Live Events How

Start coding or generate with AI.

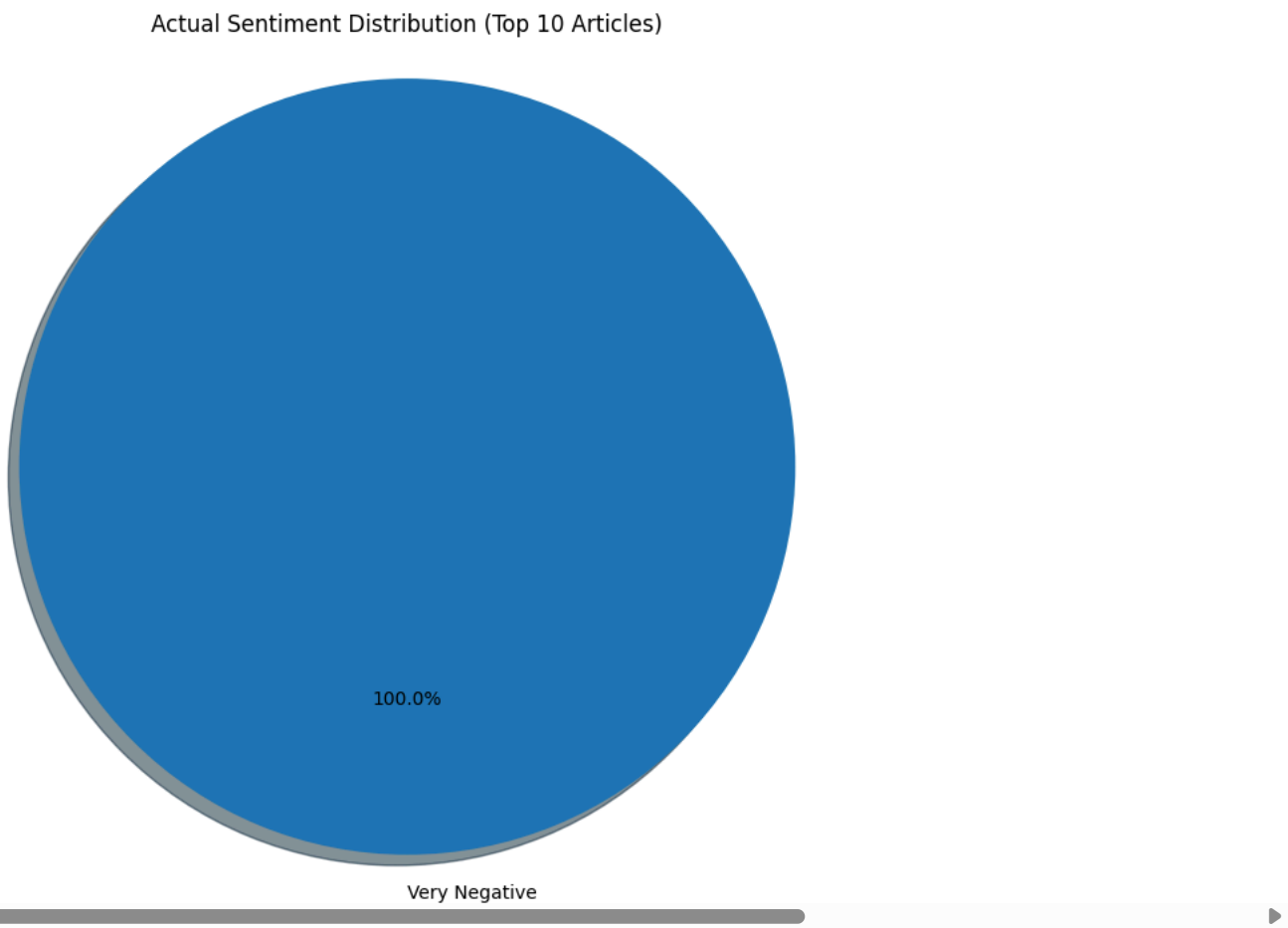
```
results_df = pd.DataFrame(results)
print(results_df[['Actual Sentiment']].head(10))
```

```
   Actual Sentiment
0              0.04
1              0.73
2              0.13
3              0.39
4             19.37
5              0.07
6              4.58
7              1.93
```

```
import matplotlib.pyplot as plt

# Prepare data
df_bar = results_df.head(10).copy()
df_bar['Label'] = ['Article ' + str(i+1) for i in df_bar.index]

# Plot horizontal bars
plt.figure(figsize=(10, 6))
bars = plt.barh(df_bar['Label'], df_bar['Actual Sentiment'], color='skyblue')
plt.xlabel('Sentiment Score (0–100)')
plt.title('Actual Sentiment Score per Article')
plt.xlim(0, 100)
plt.grid(axis='x', linestyle='--', alpha=0.5)

# Add value labels
for bar in bars:
    plt.text(bar.get_width() + 1, bar.get_y() + bar.get_height()/2,
             f'{bar.get_width():.2f}', va='center')

plt.tight_layout()
plt.show()
```

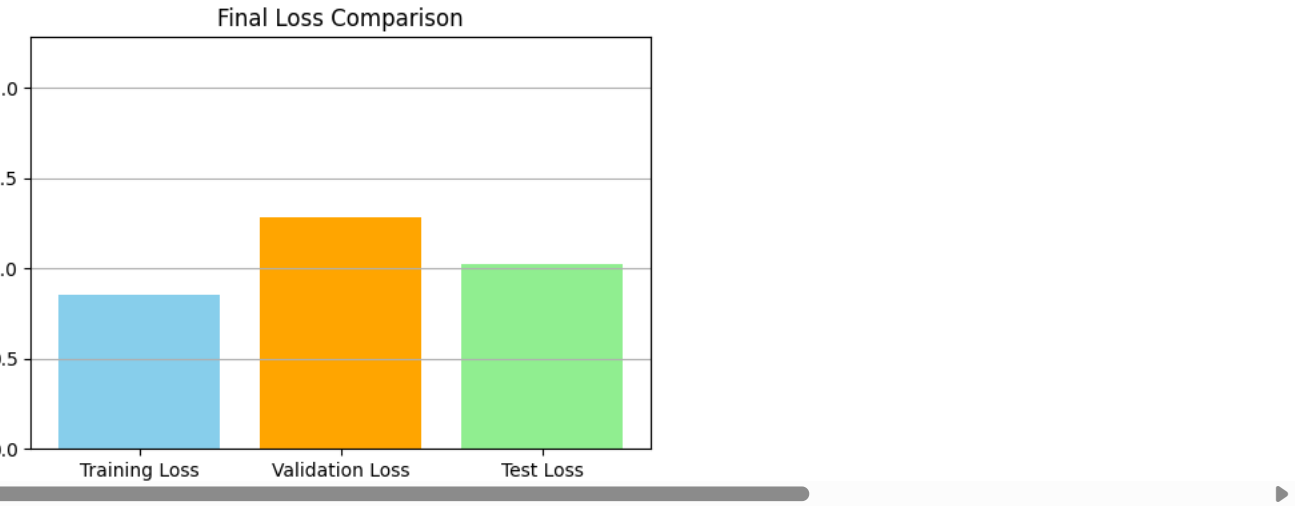



```
# Actual Sentiment Pie Chart
df_actual = results_df.head(10).copy()

def categorize(score):
    if score >= 80:
        return 'Very Positive'
    elif score >= 60:
        return 'Positive'
    elif score >= 40:
        return 'Neutral'
    elif score >= 20:
        return 'Negative'
    else:
        return 'Very Negative'

df_actual['Category'] = df_actual['Actual Sentiment'].apply(categorize)
counts = df_actual['Category'].value_counts()

plt.figure(figsize=(7, 7))
plt.pie(counts, labels=counts.index, autopct='%1.1f%%',
        startangle=90, explode=[0.03]*len(counts), shadow=True)
plt.title('Actual Sentiment Distribution (Top 10 Articles)')
plt.axis('equal')
plt.tight_layout()
plt.show()
```




```
# ✅ Evaluate average test loss
if test_losses:
    avg_test_loss = sum(test_losses) / len(test_losses)
    print(f"\n✅ Average Test Loss on {len(test_losses)} articles: {avg_test_loss:.4f}")

    # 📊 Compare with training and validation
    loss_labels = ['Training Loss', 'Validation Loss', 'Test Loss']
    loss_values = [train_losses[-1], eval_losses[-1], avg_test_loss]

    plt.figure(figsize=(6, 4))
    plt.bar(loss_labels, loss_values, color=['skyblue', 'orange', 'lightgreen'])
    plt.ylabel("Loss")
    plt.title("Final Loss Comparison")
    plt.grid(True, axis='y')
    plt.ylim(0, max(loss_values) + 1)
    plt.show()
else:
    print("⚠️ No valid test articles fetched or processed. Please check API key or try again later.")
```

```
✅ Average Test Loss on 8 articles: 1.0254
```




```
import torch
import re
```

```
import ipywidgets as widgets
from IPython.display import display, HTML

# === UI Components ===
text_box = widgets.Textarea(
    value="I love the new Tesla model!",
    placeholder='Type a Tesla-related opinion...',
    layout=widgets.Layout(width='50%', height='100px'),
    style={'description_width': 'initial'}
)

button = widgets.Button(description="🔍 Analyze Sentiment", button_style='success', icon='search')
output = widgets.Output()

# === Prediction Function (uses existing fine-tuned model & tokenizer) ===
def predict_sentiment(user_input):
    prompt = (
        "### Instruction:\n"
        "Rate the sentiment from 0 (very negative) to 100 (very positive).\n"
        "### Input: I hate this product.\n### Response: 5\n"
        "### Input: I love everything about this.\n### Response: 95\n"
        f"### Input: {user_input}\n### Response:"
    )

    # Tokenize and move to GPU
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=512).to("cuda")

    # Generate response
    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=10,
            do_sample=False,
            eos_token_id=tokenizer.eos_token_id
        )

    decoded_output = tokenizer.decode(output_ids[0], skip_special_tokens=True)

    # Strip out the prompt if included in response
    generated_text = decoded_output.replace(prompt, '').strip()

    # Extract first numeric score (0-100)
    match = re.search(r'\d+(\.\d+)?', generated_text)
    score = float(match.group()) if match else None

    return generated_text, score

# === Button Click Handler ===
def on_click(b):
    output.clear_output()
    user_input = text_box.value.strip()

    with output:
        if not user_input:
            display(HTML("<span style='color:red;'>Please enter some text.</span>"))
            return

        result_text, score = predict_sentiment(user_input)

        if score is None:
            display(HTML(f"<b>Model Response:</b> {result_text}"))
            display(HTML("<span style='color:red;'>❌ Could not extract sentiment score.</span>"))
        else:
            color = 'red' if score < 40 else 'orange' if score < 60 else 'limegreen'
            display(HTML(f"""
                <div style="font-size:18px;">
                    <b>🔍 Predicted Sentiment Score:</b> <span style="color:{color}; font-weight: bold;">{score}</span><br>
                    <b>📩 Raw Model Output:</b> {result_text}
                </div>
                <div style="margin-top:10px; background:#eee; border-radius:8px; overflow:hidden; height:24px; width:50%;">
                    <div style="width:{score}%; background:{color}; height:100%; text-align:right; padding-right:10px; color:#000;">
                        {score:.2f}%
                    </div>
                </div>
            """))

button.on_click(on_click)

# === Display UI ===
display(widgets.VBox([
    widgets.HTML("<h3 style='color: mediumvioletred;'>💡 Fine-Tuned LLaMA Sentiment Analyzer</h3>"),
    text_box,
```

```
    button,
    output
]))
```

💡 **Fine-Tuned LLaMA Sentiment Analyzer**

I hate the new Tesla model!

🔍 Analyze Senti...

🔍 **Predicted Sentiment Score: 10.0**

📩 **Raw Model Output:** 10 ### Input: I love

10.00%

```
import torch
import re
import ipywidgets as widgets
from IPython.display import display, HTML

# === UI Components ===
text_box = widgets.Textarea(
    value="I love the new Tesla model!",
    placeholder='Type a Tesla-related opinion...',
    layout=widgets.Layout(width='50%', height='100px'),
    style={'description_width': 'initial'}
)

button = widgets.Button(description="🔍 Analyze Sentiment", button_style='success', icon='search')
output = widgets.Output()

# === Prediction Function (uses existing fine-tuned model & tokenizer) ===
def predict_sentiment(user_input):
    prompt = (
        "### Instruction:\n"
        "Rate the sentiment from 0 (very negative) to 100 (very positive).\n"
        "### Input: I hate this product.\n### Response: 5\n"
        "### Input: I love everything about this.\n### Response: 95\n"
        f"### Input: {user_input}\n### Response:"
    )

    # Tokenize and move to GPU
    inputs = tokenizer(prompt, return_tensors="pt", truncation=True, max_length=512).to("cuda")

    # Generate response
    with torch.no_grad():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=10,
            do_sample=False,
            eos_token_id=tokenizer.eos_token_id
        )

    decoded_output = tokenizer.decode(output_ids[0], skip_special_tokens=True)

    # Strip out the prompt if included in response
    generated_text = decoded_output.replace(prompt, '').strip()

    # Extract first numeric score (0-100)
    match = re.search(r'\d+(\.\d+)?', generated_text)
    score = float(match.group()) if match else None

    return generated_text, score

# === Button Click Handler ===
def on_click(b):
    output.clear_output()
    user_input = text_box.value.strip()

    with output:
        if not user_input:
            display(HTML("<span style='color:red;'>Please enter some text.</span>"))
            return

        result_text, score = predict_sentiment(user_input)

        if score is None:
```

```
            display(HTML(f"<b>Model Response:</b> {result_text}"))
            display(HTML("<span style='color:red;'>❌ Could not extract sentiment score.</span>"))
        else:
            color = 'red' if score < 40 else 'orange' if score < 60 else 'limegreen'
            display(HTML(f"""
                <div style="font-size:18px;">
                    <b>🔍 Predicted Sentiment Score:</b> <span style="color:{color}; font-weight: bold;">{score}</span><br>
                    <b>📩 Raw Model Output:</b> {result_text}
                </div>
                <div style="margin-top:10px; background:#eee; border-radius:8px; overflow:hidden; height:24px; width:50%;">
                    <div style="width:{score}%; background:{color}; height:100%; text-align:right; padding-right:10px; color:#000;">
                        {score:.2f}%
                    </div>
                </div>
            """))

button.on_click(on_click)

# === Display UI ===
display(widgets.VBox([
    widgets.HTML("<h3 style='color: mediumvioletred;'>💡 Fine-Tuned LLaMA Sentiment Analyzer</h3>"),
    text_box,
    button,
    output
]))
```

💡 **Fine-Tuned LLaMA Sentiment Analyzer**

I love the new Tesla model!

🔍 Analyze Senti...

🔍 **Predicted Sentiment Score: 90.0**

📩 **Raw Model Output:** 90 ### Input: I hate

90.00%

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.